

I. Executable and Linkable Format (ELF)^[1]

I-a) ELF Header

File Class `e_ident[EI_CLASS] = (ELFCLASS32 | ELFCLASS64)`

Data Encoding `e_ident[EI_DATA] = ELFDATA2LSB`

Processor Identification `e_machine = EM_X86_64`

#entry of program header table (`e_phnum` | `sh_info` of section header at index 0)

`e_phnum = PN_XNUM` → `sh_info` if `e_phnum ≥ PN_XNUM` else `sh_info = 0` → `e_phnum`

Size of program header table `e_phnum * e_phentsize`

I-b) Sections

> **2GB section mark** `sh_flags = SHF_X86_64_LARGE` (AMD64 Specific Section Header Flag)

Name	Type	Attributes
<code>.got</code>	<code>SHT_PROGBITS</code>	<code>SHF_ALLOC+SHF_WRITE</code>
<code>.plt</code>	<code>SHT_PROGBITS</code>	<code>SHF_ALLOC+SHF_EXECINSTR</code>
<code>.eh_frame</code>	<code>SHT_X86_64_UNWIND</code>	<code>SHF_ALLOC</code>

1) Suggested section ordering

1. `.plt .init .fini .text .got .rodata .rodata1 .data .data1 .bss` (total size up to 2GB)
2. `.lplt .ltext .lgot .lrodata .lrodata1 .ldata .ldata1 .lbss` (up to 16EB, for large code model)

2) Symbol Table

1. Optional `STT_GNU_IFUNC` symbol type same as `STT_FUNC` except always pointing to a function or piece of executable code which takes no arguments and returns function pointer
2. If an `STT_GNU_IFUNC` symbol is referred to by a relocation, then evaluation of that relocation is delayed until load-time and the value used in the relocation is the function pointer returned by invocation of the `STT_GNU_IFUNC` symbol
3. `STT_GNU_IFUNC` allows run-time to select between multiple versions of implementations

3) Global Offset Table `.got`

1. Redirect position-independent address calculations to absolute locations
2. A symbol is required direct access to absolute address of will have a GOT entry
3. Since executable or shared object has its own GOT, a shared symbol may have entries in multiple tables. Dynamic linker populates these GOT relocations before process execution

4. Entry GOT[0] is reserved for `_DYNAMIC` structure address, allowing dynamic linker to locate its own metadata and self-initialize (without relying on other programs to relocate its memory image) before relocations run. On AMD64, GOT[1] and GOT[2] are also reserved
5. `_GLOBAL_OFFSET_TABLE_` may reside in middle of `.got`, allowing (non) negative offsets

4) Procedure Linkage Table .plt

1. Redirect position-independent function calls to absolute locations
2. **Problem** `main.c` sees `&func == 0x401000` but `lib.so` sees `&func == 0x7FFFF7A03000`
 1. References from within shared objects will be resolved by dynamic linker to virtual address of the function itself but from within executable to a function in a shared object will be resolved by link editor to address of PLT entry for that function within executable
 2. Link editor will place address of PLT entry for that function in its associated symbol entry (with section index of `SHN_UNDEF` but a type of `STT_FUNC` and `st_value` $\neq 0$)
 3. When a shared object needs to take the address of a function (i.e. creating a function pointer), the dynamic linker redirects that lookup to the main executable's symbol table entry instead of using the function's real location inside the shared object
3. Link editor cannot resolve execution transfers (such as function calls) from executable or shared object to another, thus it arranges to have program transfer control to PLT's entries
4. On AMD64, PLTs reside in shared text, but they use addresses, determined by dynamic linker, in private GOT
5. Executable files and shared object files have separate PLTs

5) Unwinding The Stack

1. **Reason** language-specific exceptions and "forced" unwinding (e.g. `longjmp`, thread termination)
2. **Rule**
 1. As stack is unwound while exception propagates, each frame's (language and vendor specific) personality routine decides whether to catch or pass through exception - whether it is "native" or "foreign"
 2. During forced unwinding, an external agent (like `longjmp`), not each personality routine, decides when to stop. The `_UA_FORCE_UNWIND` flag explicitly signals that unwinding must continue and cannot be stopped by the personality routine
 3. While `_Unwind_RaiseException` relies on frame personality routines to decide how exceptions are handled, `_Unwind_ForcedUnwind` uses a proxy personality routine to intercept those calls and let an external agent override their default decisions
 4. CFI - Call Frame Information - needed for unwinding the stack is output into one or more ELF sections of type `SHT_X86_64_UNWIND`, such as `.eh_frame`

5.1) 2-phase Process (after raising exception)

1. Search Phase

1. Repeatedly calls personality routine, with `_UA_SEARCH_PHASE` flag, for current `%rip` and register state, then unwinding frame to new `%rip`, until personality routine reports either success (handler found in queried frame) or failure (no handler) in all frames
2. It does not actually restore unwound state (e.g., real hardware registers are not modified, no stack frames are popped, and no destructors are executed yet) and personality routine must access state through API

2. Cleanup Phase

1. Failure $\Rightarrow$ `terminate()`
2. Success $\Rightarrow$ cleanup - repeatedly calls personality routine, with `_UA_CLEANUP_PHASE` flag, first for current `%rip` and register state, and then unwinding frame to new `%rip`, until it gets to frame with an identified handler
3. Once personality routine prepares (override) registers (`%rdi`, `%rsi`, `%rdx`, `%rcx`) for landing pad parameters and returns `_URC_INSTALL_CONTEXT`, unwind library uses context record to restore register state, callee-saved register and 4 registers above, to that in frame before the call that threw exception
4. Finally, control is transferred to user landing pad code - that can either resume normal execution (e.g., at end of a C++ catch), or `_Unwind_Resume` (will never return) and passing it the `exceptionObject` argument received by personality routine

3. NOTE

1. `_Unwind_Resume` iff personality routine didn't return `_Unwind_HANDLER_FOUND` during phase 1. This allows unwinder to track allocated resources (must be freed before entering a handler landing pad, but not before non-handler pads, since `_Unwind_Resume` will eventually handle cleanup) in exception object
2. Each phase uses both unwind library - for locating and restoring previous stack frames and personality routines - for handler and mechanism for transferring control to it
3. First phase allows to dismiss exception before stack unwinding begins, which allows non-C++ presumptive exception handling (correcting exceptional condition and resuming execution at the point where it was raised)
4. CFA - Canonical Frame Address - the value of `%rsp` at the call site in the previous frame and LSDA means Language-Specific Data Area
5. A catch-all block may be executed during forced unwinding. If happens, **unwinding will proceed** at end of catch-all block, whether or not there is an explicit re-throw

5.2) Unwind Function Table `.eh_frame`

1. Consist of subsections, each contains a CIE followed by varying number of FDEs

2. FDE corresponds to explicit or compiler generated function in a compilation unit, i.e. a separate FDE for each contiguous sub-piece in function code that is not one contiguous block, all FDEs can access the CIE that begin their subsection for data
3. For C++ template instantiations, there shall be a separate CIE immediately preceding each FDE corresponding to an instantiation
4. Using EH_PE encoding - pointer encoding, .eh_frame can be entirely resolved at link time, thus can become part of text segment
5. The existence and size of optional call frame instruction area must be computed based on overall size and offset reached while scanning preceding fields of CIE or PDE
6. Overall size of .eh_frame is given in ELF section header, but number of entries is only determined thru counting!
7. .eh_frame whose format is identical to .debug_frame defined by DWARF with extensions:
 1. **Position independence** avoid load time relocations for PIC, FDE CIE offset pointer should be stored relative to start of CIE table entry
 2. **Outgoing arguments area delta** using GNU_ARGS_SIZE to maintain size of temporarily allocated outgoing arguments area present on end of stack
 3. **CIE Augmentations** using a PC relative encoding whenever possible and adjusting size according to code model used

5.2.1. Common Information Entry (CIE)

Field	Description
Length	Length of the CIE (not including this field)
CIE id	0 for .eh_frame (distinguish CIE and FDE)
Version	1
CIE Augmentation String	Null-terminated string (e.g., "zR", "zPLR", or "")
Code Align Factor	To be multiplied with "Advance Location" instructions in the Call Frame Instructions
Data Align Factor	To be multiplied with all offsets in Call Frame Instructions
Ret Address Reg	"Virtual" register representation of return address
CIE Augmentation Section	(Optional) Present if CIE Augmentation String is non empty
Call Frame Instructions	(Optional)

CIE Augmentation Section Content

Char	Operands	Description
z	size	Length of remainder of Augmentation Section
P	personality_enc	Encoding specifier - preferred value is a pc-relative
	personality_routine	Encoded pointer to personality routine (actually to the PLT entry for personality routine)
R	code_enc	Non-default encoding for code-pointers (FDE's initial_location, address_range and operand for DW_CFA_set_loc) - preferred value is pc-relative
L	lsda_enc	FDE augmentation bodies may contain LSDA pointer. Its preferred value is pc-relative possibly indirect thru a GOT entry

5.2.2. Frame Descriptor Entry (FDE)

Field	Description
Length	Length of the FDE (not including this field)
CIE pointer	Distance from this field to nearest preceding CIE that can never be 0 (distinguish CIE and FDE)
Initial Location	Reference to corresponded function code
Address Range	Size of the function code
FDE Augmentation Section	(Optional) Present if CIE Augmentation String is non empty
Call Frame Instructions	(Optional)

Pointer Encoding Specification Byte

Mask	Meaning
0x1	Values are stored as <code>uleb128</code> or <code>sleb128</code> type (according to flag 0x8)
0x2	Values are stored as 2 bytes wide integers (<code>udata2</code> or <code>sdata2</code>)
0x3	Values are stored as 4 bytes wide integers (<code>udata4</code> or <code>sdata4</code>)
0x4	Values are stored as 8 bytes wide integers (<code>udata8</code> or <code>sdata8</code>)
0x8	Values are signed
0x10	Values are PC relative
0x20	Values are text section relative
0x30	Values are data section relative
0x40	Values are relative to the start of function

5.3) Unwind Through Assembler Code

.cfi_startproc begin of each function that should have an entry in `.eh_frame` (initializes some internal data structures and emits architecture dependent initial CFI instructions)

.cfi_endproc end of a function

.cfi_def_cfa REGISTER, OFFSET CFA = REGISTER's address + OFFSET

.cfi_def_cfa_register REGISTER CFA = REGISTER's address + old OFFSET

.cfi_def_cfa_offset OFFSET CFA = old REGISTER's address + OFFSET

.cfi_adjust_cfa_offset OFFSET CFA = old REGISTER's address + (old += OFFSET)

.cfi_offset REGISTER, OFFSET saves previous REGISTER value at offset OFFSET from CFA

.cfi_rel_offset REGISTER, OFFSET saves previous REGISTER value at offset OFFSET from current CFA register

.cfi_escape EXPRESSION[, ...] allows user to add arbitrary bytes to the unwind info (such as OS-specific CFI opcodes, or generic CFI opcodes that assembler doesn't support)

I-d) Auxiliary Vector

```
typedef struct {  
    int a_type;  
    union { long a_val; void *a_ptr; void (*a_fnc)(); } a_un;  
} auxv_t;
```

0. **AT_NULL** – marks the last entry
1. **AT_IGNORE** – entry has no meaning
2. **AT_EXECD** – file descriptor open to read the application program's object file
3. **AT_PHDR** – address of the program header table in the memory image
4. **AT_PHEENT** – size of each program header entry
5. **AT_PHNUM** – number of program header entries
6. **AT_PAGESZ** – system page size in bytes
7. **AT_BASE** – base address where the interpreter was loaded
8. **AT_FLAGS** – one-bit flags
9. **AT_ENTRY** – program's entry-point
10. **AT_NOTELF** – non-zero if program is not ELF
11. **AT_UID** – real user ID
12. **AT_EUID** – effective user ID
13. **AT_GID** – real group ID
14. **AT_EGID** – effective group ID
15. **AT_PLATFORM** – platform name string
16. **AT_HWCAP** – CPU hardware-feature bitmask
17. **AT_CLKTCK** – frequency of times()
...
23. **AT_SECURE** – non-zero if running in secure mode (e.g. setuid)

- 24. **AT_BASE_PLATFORM** – base architecture platform name
- 25. **AT_RANDOM** – pointer to 16 securely generated random bytes
- 26. **AT_HWCAP2** – additional CPU-feature bitmask
- ...
- 31. **AT_EXECFN** – pointer to the executed program's file name

II. TRIVIAL

II-a) Debug With Arbitrary Record Format

- 1. DWARF is a specification developed for symbolic, source-level debugging
- 2. The debugging information format does not favor the design of any compiler or debugger

[1]: [System V ABI](#)